

Real-Time Railway Obstacle Detection Based on Multitask Perception Learning

Chenglin Chen^{ID}, Huixiong Qin^{ID}, Yong Qin^{ID}, *Senior Member, IEEE*, and Yun Bai^{ID}

Abstract—Railway object intrusion poses a significant threat to railway safety, so it is vital to monitor the obstacles within the track area in real-time to prevent accidents, which can be achieved by vision-based technologies. However, most existing vision-based railway obstacle detection algorithms are limited to stationary cameras, which restricts the monitoring range. In contrast, onboard approaches enable full-line monitoring but face more challenges in achieving high accuracy and real-time performance. To address these challenges, we developed an onboard end-to-end multitask perception model (MTP-Rail), which includes variants of different scales (S, M, L) to realize railway obstacle detection in real-time with high accuracy. Our model consists of two decoders, which can simultaneously implement the tasks of object of interest detection and track segmentation. In addition, we designed a post-processing scheme to analyze the relationship between the detected object and the track and assess the obstacle risk level. Experiments on our dataset show that one of the proposed model MTP-rail-M achieved an 89.6% classification accuracy of obstacle risk level and 54.3% on mAP, 78.6% on mPA and 64.9% on mIoU with an inference speed of 181 FPS at an input size of 640×640 on RTX 3060Ti. Our model is easy to install and deploy, highlighting its potential in engineering applications. The code is available on https://github.com/ccl-1/obstacle_detection.

Index Terms—Multitask, obstacle detection, semantic segmentation, railway intrusion monitoring, railway safety.

I. INTRODUCTION

IN RECENT years, urban rail transit has become an essential component of modern transportation systems with its efficiency, safety, and environmental friendliness. However,

as the scale of rail transit systems continues to expand, the requirements for their safety and reliability are becoming increasingly stringent, especially in the complex and ever-changing operating environment. The presence of obstacles in the track area poses potential threats to railway safety, particularly in enclosed systems such as subways and light rails. For instance, obstacles may cause trains to derail or overturn, possibly resulting in casualties and property loss. The key to preventing accidents and ensuring train safety is to detect potential obstacles early and accurately [1]. Railway obstacle detection is a method aimed at detecting obstacles within the railway track area. These obstacles may include but are not limited to trains, pedestrians, animals, billboards, fallen rocks, and trees, which can threaten railway safety. However, timely and accurately detecting obstacles is difficult, especially in complex urban traffic environments.

Early obstacle detection was typically performed through manual observation and inspections by train drivers, maintenance personnel, etc. Nevertheless, with the progress of computer vision (CV) and artificial intelligence (AI), automated obstacle detection systems are gradually being used in intelligent railway transportation systems [2], [3]. Railway obstacle detection systems generally utilize sensors like cameras and LiDAR to capture images or point cloud data of railway scenes, which are then analyzed to detect obstacles. Specifically, it consists of three key steps: interest object detection, track area extraction, and the post-processing to analyze the risk between interest objects and track area. External objects within the track area will be regarded as obstacles. Even with these blueprint and algorithmic paradigm, railway obstacle detection remains a major challenge. Current methods of railway obstacle detection primarily fall into two categories: traditional CV-based and deep learning (DL)-based. Traditional CV-based methods mainly rely on conventional computer vision and image processing methods. Such as extracting railway track area and interest objects by edge detection algorithms (e.g., Canny [4] and Hough [5]) while performing filters to denoise images. However, such approaches highly depend on hand-crafted features and fail to address complex and changeable scenes. In other words, traditional CV-based methods lack strong robustness against external changes in the operating environment. Recently, researchers have shown an increasing interest in using AI and DL to detect railway obstacles. Some classic one-stage [6], [7] or two-stage [8] algorithms have been successfully applied and verified in autonomous driving, industrial inspection, etc.

Received 15 August 2024; revised 1 December 2024; accepted 24 January 2025. Date of publication 13 February 2025; date of current version 5 May 2025. This work was supported in part by the State Key Laboratory of Rail Traffic Control and Safety, Beijing Jiaotong University, under Contract RCS2023K007; in part by Guangzhou Municipal Science and Technology Projects under Grant 2023A03J0011 and Grant 2024A03J0613; in part by Guangdong Provincial Key Laboratory of Integrated Communication, Sensing and Computation for Ubiquitous Internet of Things under Grant 2023B1212010007; and in part by the Tertiary Education Scientific Research Project of Guangzhou Municipal Education Bureau Project under Grant 2024312039. The Associate Editor for this article was Q. He. (*Corresponding authors: Yun Bai; Yong Qin.*)

Chenglin Chen is with the Thrust of Intelligent Transportation, The Hong Kong University of Science and Technology (Guangzhou), Guangzhou 511400, China (e-mail: cchen363@connect.hkust-gz.edu.cn).

Huixiong Qin is with the Center for Advanced Infrastructure and Transportation, Rutgers University, New Brunswick, NJ 08854 USA (e-mail: hq63@soe.rutgers.edu).

Yong Qin is with the State Key Laboratory of Rail Traffic Control and Safety, Beijing Jiaotong University, Beijing 100044, China (e-mail: yqin@bjtu.edu.cn).

Yun Bai is with New Jersey Institute of Technology, Newark, NJ 07102 USA (e-mail: yun.bai@njit.edu).

Digital Object Identifier 10.1109/TITS.2025.3536024

DL-based methods demonstrate superior adaptability in complex and dynamic environments due to their powerful feature learning and representation capabilities. As a data-driven algorithm, deep learning has been widely used in autonomous driving, largely due to the availability of large data sets such as COCO [9], nuScenes [10], KITTI [11], and others. Large data sets provide sufficient samples for the perception and understanding of the environment, enabling the model to learn more generalized representations. However, compared with road traffic, the railway system is relatively closed and suffers from insufficient data, which restricts the development of intelligent railway obstacle detection to a large extent. Up to now, much of the research on DL-based railway obstacle detection has been involved in model structure modification, parameter tuning, retraining based on mainstream general detection (e.g., SSD [6], YOLO [7], RetinaNet [12], Faster-RCNN [8]) or segmentation (e.g., Mask-RCNN [13], BiSeNet [14]) models to fit the new railway data.

In rail transit, many of the previous studies treat railway obstacle detection as railway object detection and focus on improving detection accuracy, speed, and robustness to complex environments. The optimization strategy mainly includes data augmentation [15], [16], [17], feature fusion, multi-scale fusion [18], attention mechanism [19], model structure modification, etc. Besides, [20] innovatively proposes an early warning approach that fuses the detection of events and information on foreign objects severities. It is worth noting that railway obstacle detection actually involves three steps as mentioned before, yet most researchers focus on either step 1 or step 2, lacking unified approaches to the entire process that analyzes the relationship between objects and track area. Some studies, such as [21], have implemented a complete railway obstacle detection pipeline. These algorithms usually use two independent models: one for object detection and the other for track region segmentation. Although these algorithms have basic obstacle detection functions, they require a lot of computing resources and high hardware performance, which brings challenges to model installation and deployment. In addition, due to the insufficiency of railway data, especially data related to obstacle detection, these algorithms are either experimented on private datasets or lack of validation of the effectiveness of intrusion detection, which makes it difficult to assess their applicability in real-world applications.

More recently, there have been rapid advances in multi-sensor fusion. Some researchers have attempted to fuse multiple sensor data to improve the performance of railway obstacle detection [22], [23], [24]. Among them, RGB and LiDAR fusion has become a standard method. Multi-sensor fusion has the potential to shorten the detection distance and enhance accuracy, while it also faces several challenges, such as calibration complexity, heterogeneous data integration, and real-time requirements. For example, to ensure safety, prompt obstacle detection on the track requires the multi-sensor fusion algorithm to process data from multiple sensors in real-time, which poses significant challenges. Because the general multi-sensor fusion algorithm is highly complex and needs

vast computing resources, achieving real-time performance on embedded systems or devices with constrained computing resources is difficult. So far, the multi-sensor fusion method is still in the early stage of application in rail transit, and the solution has not reached a mature level in terms of cost, stability, real-time deployment, and so on.

Considering all these challenges and research gaps, we aim to develop a purely visual multitask-based end-to-end railway obstacle detection system that does not rely on LiDAR or any supplementary data from other active sensors. The specific objective of this study was to achieve rapid and accurate assessment of potential risks of obstacles within the railway track area by analyzing the image and video data collected by the forward-facing camera. To the best of our knowledge, the proposed approach is the first open-source end-to-end railway obstacle intrusion detection model based on multitask perception. Our main academic contributions are summarized as follows.

- We proposed an end-to-end multitask railway obstacle detection model that can simultaneously complete the tasks of interest object detection, track area segmentation, and obstacle analysis. It not only achieves high detection and segmentation accuracy but also runs in real-time and is easy to deploy.
- We designed a post-processing scheme to assess the level of risk for detected obstacles.
- We collected and annotated a small dataset for multi-task railway obstacle detection and auxiliary datasets for transfer learning. We have made these datasets publicly available to support other relevant research.
- We conducted extensive experiments to validate the effectiveness of our model and explored different model architectures and training strategies for optimal detection and segmentation tasks.

Practically, this study aims to find an efficient and practical obstacle detection solution for rail transit systems so as to improve their intelligence level, safety, and operation efficiency. Our algorithm features real-time monitoring and alert capabilities, enabling it to identify potential safety hazards promptly. This reduces the workload of train staff and significantly lowers the risk of train delays and safety incidents caused by obstacle intrusions, thereby comprehensively enhancing the efficiency and safety of railway operations. Additionally, automatic train systems require high-precision, real-time environmental perception, and obstacle detection capabilities to ensure safe operation without human intervention. Our multitask railway obstacle detection system can provide technical support to extensively promote the intelligent and automated level of the railway transportation system.

The remaining part of the paper proceeds as follows. Section II reviews related research and describes the differences between our approach and existing methods. Section III presents the detailed methodology developed in this study. Extensive experiments and analysis are conducted in Section IV to evaluate the proposed method. Finally, we conclude the paper and discuss future work.

TABLE I
PREVIOUS STUDIES IN VISION-BASED RAILWAY OBSTACLE DETECTION

Method	Ref.	Description	Limitations
Traditional CV-based	[26]	1) Detect tracks by Hough transform and warp it into track-parallel images by projective transform; 2) Find obstacles by analyzing the relationship between tracks in a designated search region.	1) High false alarm rate, and poor robustness; 2) Suitable only for scenarios with simple backgrounds; 3) More challenging datasets and thorough experiments should be used to evaluate the model.
	[27]	1) Extract rail track using starting point and adaptive sliding windows; 2) Detect obstacles by analyzing the angle between tracks in adjacent windows.	
	[28]	Extract tracks and objects by threshold segmentation and edge detection.	
	[29]	Obstacles are detected through background subtraction between the real-time image and pre-collected image aligned with it.	
DL-based	Stationary camera	[30] Propose enhanced few-shot learning for intrusion detection in railway video surveillance.	1) With stationary cameras, only part of the area can be monitored; 2) Lack of experimental validation of track intrusion.
		[31] Explore MobileNetv2 [32] for obstacle detection at rail crossings through data augmentation and transfer learning.	
		[21] 1) Propose a mountainous railway track monitoring system based on Yolov5 [33] and BiSeNetv2 [34]; 2) Detect obstacles on the railbed, such as avalanches, mudslide.	
	Onboard camera	[35] 1) Propose an embedded system for multi-scale object detection; 2) Introduce feature fusion and lightweight feature extraction strategies.	1) Devote to small object detection but not obstacle detection; 2) High data similarity in the dataset leads to overfitting.
		[36] 1) Extract possible foreign presence frames by median filtering and gaussian mixture model; 2) YOLOv3 [37] for object detection.	Only applies to moving target and treats all objects in the scene as obstacles.
		[38] 1) Combine object detection and semantic segmentation to detect obstacles and classify them into three levels base on the regions they occupy; 2) Propose post-processing methods for track lane segmentation and unseen obstacle detection.	Running both object detection and semantic segmentation leads to computational overheads.
	Light model	[39] 1) Split the model to achieve collaborative inference and split the computational load; 2) The ground server and the train can coordinate inferences.	There are communication transmission delays of features and information security hazards.
		[40] 1) Use a coarse region proposal module to generate top-5 candidate sub-images; 2) Employ a lightweight model with attention mechanism to detect objects.	1) The improvement is not significant compared with the YOLO-series; 2) Speed increase comes at the expense of huge accuracy decrease.
		[41] 1) Use morphology and background difference for coarse intrusion localization; 2) Lightweight model for fine object detection.	1) Intrusion region extraction requires hand-crafted parameters; 2) Suitable only for fixed surveillance video scenes.
		[18] 1) Propose a model with multi-scale augmentation for high-speed railway clearance scene; 2) Adopt bidirectional iterative deep aggregation and spatial pyramid pooling for fuse feature.	Unable to run in real-time and not easy to deploy.
	Image generation	[42] 1) Explore unsupervised models to detect obstacles on railway track; 2) Normal and abnormal images are distinguished by reconstruction scores.	More challenging datasets and thorough experiments should be studied to evaluate the model.
		[43] 1) Generate obstacle-free railway images with a generative model; 2) Use semantic differentiator to locate obstacles; 3) Focus on distinct and repetitive textures, suitable for highly structured scenes.	1) Perform unreliable on complex backgrounds; 2) The experiment only covers the evaluation of the generation part.
	Multitask learning	[44] 1) A multitask intrusion detection model; 2) Object and track line detection by row classification.	1) Not an end-end model, tasks are executed serially; 2) The object and track line detection results are simply added together without analyzing whether the object is an obstacle.

II. RELATED WORKS

A. Obstacle Detection

Recently, many researchers have paid attention to railway obstacle detection. Traditional CV-based, DL-based, and mixed methods are three popular types of approaches to achieve it [1], [25]. Different techniques have contributed to object detection, track area, or track line extraction by improving the model accuracy, speed, robustness, and detection

distance. Table I summarizes the vision-based railway obstacle detection approaches as follows.

Traditional CV-based railway obstacle detection methods utilize edge detection, threshold segmentation, background subtraction, and morphological processing to extract railway tracks or objects of interest. These approaches are commonly used in video surveillance and scenarios with relatively less complex backgrounds and rely heavily on hand-crafted parameters for feature extraction. The application of DL

has significantly improved the model's ability to learn and represent features. Some scholars have designed lightweight detection or segmentation networks to improve inference speed [35], [38], [39], [40], [41]. Others have addressed the challenges of small objects and multi-scale issues through techniques such as data augmentation [15], [16], [17], multi-scale fusion [18], attention modules [19], etc. Additionally, some mixed methods have been employed, namely using traditional CV methods to extract moving objects or track areas and DL-based methods to recognize objects in images [36], [41]. In particular, some researchers use a generator to generate artificial images without obstacles which are used as benchmarks to distinguish the images with obstacles [42], [43]. However, the stability and feasibility of their adaptive applications remain to be verified.

Railway-related image data is generally obtained from either onboard or stationary surveillance cameras. Some scholars have devoted themselves to developing obstacle detection in stationary surveillance camera scenarios [21], [30], [31], [45]. Since the scene captured by stationary surveillance cameras is fixed, obstacle extraction is relatively easy. For example, background differences can separate the background and the target. However, monitoring the entire line using stationary cameras requires many devices, and is subject to the obvious limitations in transmission, processing and analysis of massive amount of data, and equipment maintenance. A few scholars [36], [44] are committed to developing onboard railway obstacle detection, which is also our research subject. The scene changes greatly in this case, so that the traditional algorithms perform poorly. As mentioned above, the conventional implementation of onboard visual railway obstacle detection requires three steps: interest object detection, track area extraction, and the post-processing of the interest objects and track area. However, most existing algorithms only cover the first two parts. Some methods realize railway object detection and track segmentation but do not further subdivide the detected object. They assume that all objects in the scene are obstacles, which may misjudge the objects in the safe area beside the track [18], [35], [36], [39], [40], [41]. There are also some other algorithms that theoretically achieve obstacle detection, but there are no corresponding experimental results to support it [30], [43], [44].

Based on the above investigation, we found that it is necessary to develop an end-to-end multitask network to simultaneously perform obstacle detection and track area segmentation, along with a post-processing method to achieve fast and accurate recognition of obstacles within the track area.

B. Multitask

Multitask learning has broad engineering application prospects in both autonomous driving and rail transit environmental perception. In real-time railway obstacle detection, safety is the top priority, and thus speed and computational efficiency are critical to ensure prompt detection of potential hazards. Learning-based methods often treat obstacle detection and track segmentation as independent tasks during training and inference, leading to increased computational overhead. A multitask model,

on the other hand, enables end-to-end training and enhances inference speed by sharing encoded parameters across tasks, making it more suitable for practical deployment to real-time realization. Therefore, leveraging multitask learning is both necessary and vital for railway obstacle detection.

Mask RCNN [13] is a classic two-stage framework for object detection and instance segmentation. It achieves high accuracy but is time-consuming and challenging to install and deploy. YOLOP [46] accomplishes three tasks: object detection, drivable area segmentation, and lane segmentation, achieving 23 FPS at input size of 640×384 on Jetson TX2. Nevertheless, this algorithm performs well only in single-class object detection cases. HybridNets [47], similar to YOLOP, improves detection accuracy mainly by optimizing the model structure and loss function. So far, little attention has been paid to multitask learning for railways, and to the best of our knowledge, only [44] is related to railway intrusion detection. It uses a single model to perform object detection and track line segmentation based on row classification. However, this model cannot achieve real end-to-end training; in practical application, the detection and segmentation tasks have to be trained sequentially. Moreover, it simply combines the results of object and track line segmentation without explicitly distinguishing whether an object is an obstacle. Besides, it is worth mentioning that track area and track line segmentation play crucial roles in determining the intrusion boundary, yet the difficulty of these two segmentation tasks differs notably. Track lines are usually slender and occupy fewer pixels in the image, making their segmentation more complex and usually less accurate than track area segmentation. Therefore, considering that the ultimate goal of both tasks is to delineate the intrusion boundary, choosing the more complex track line segmentation is unnecessary.

Thus, this study proposes an end-to-end multitask network to achieve railway obstacle detection in real-time. Generally speaking, there are two primary innovative aspects compared with existing works: 1) Compared with previous studies on the task of railway obstacle detection, the proposed method is an end-to-end multitask network that can achieve railway obstacle detection with high accuracy and real-time performance. It is easy to deploy and thus has practical engineering application value. 2) Compared with the multitask network in autonomous driving, the proposed model performs better in railway obstacle detection. It shows high generalization ability and low latency on public autonomous driving datasets too, especially in multi-class object detection.

III. METHODOLOGY

To better detect the obstacle, we proposed an end-to-end vision-based multitask perception solution capable of real-time obstacle detection and risk level assessment. In this section, we describe the structure of the proposed model and elaborate on the differences and advantages between it and previous methods. In addition, some model optimization strategies are introduced as well.

A. Overview

Inspired by YOLOP, our model follows a similar design. YOLOP uses different segmentation heads to predict drivable

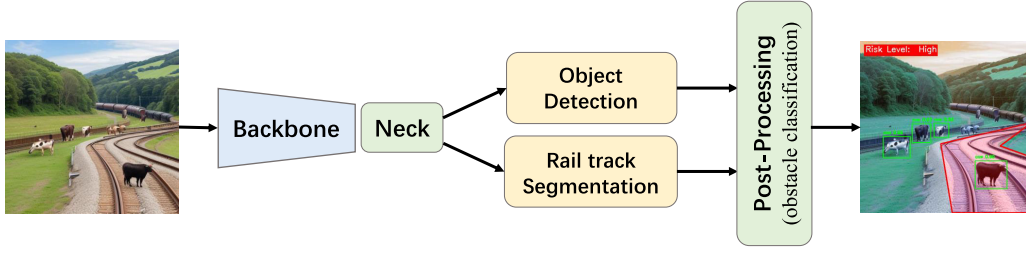


Fig. 1. Overview of the proposed model.

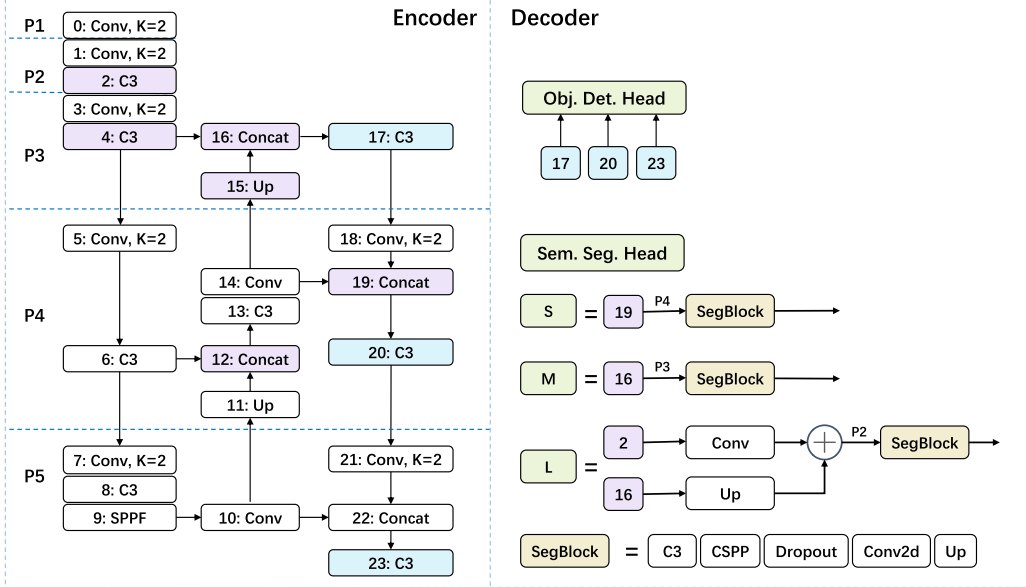


Fig. 2. Illustration of the proposed model with three segmentation decoder designs. Numbers 0-23 represent the network layers, and the C3, CSPP, and SPPF modules have the same structure as those in YOLOv5 [33].

areas and lanes separately due to the differing feature level demands of these tasks. However, this increases model complexity. In our application, one single segmentation head is sufficient since we only need to segment the track area. Overall, the proposed algorithm is a one-stage multitask detector capable of end-to-end training, detecting, and segmenting regions of interest in real-time with high accuracy. The overall architecture is shown in Fig. 1.

The proposed railway multitask perception model for obstacle detection (MTP-Rail) includes a shared encoder and two decoders to perform detection and segmentation tasks. The shared encoder consists of the backbone and neck. First, the input image is fed into the backbone for feature extraction, and then the low-level and high-level features are fused by a neck to construct multi-scale features. Next, these multi-scale features are then decoded by detection and segmentation heads to predict the track area and object of interest. Finally, these prediction results are fed into the post-processing module. In this module, we analyze the relationship between the objects and tracks, such as their relative distances, to assess the risk level of the current scene quantitatively.

B. Architecture Details

As it is mentioned before, in the context of railway multitask obstacle detection, three tasks need to be completed: 1) Detecting objects of interest, 2) Extracting railway track areas, and 3)

Assessing the risk of obstacles based on the objects and track areas. We adopt an anchor-based multi-scale detection scheme of YOLOv5 for task 1 due to its superior accuracy, low latency, and high extensibility. In simple terms, we, following YOLOv5 architecture, divide the feature maps into grids at three scales, assigning three anchors with different aspect ratios to each grid. The detection head predicts the offset of each anchor's position, the scaling of height and width, and the probability and confidence for each class.

For task 2, our implementation is straightforward, achieved through several layers of convolution and upsampling. It is worth noting that in computer vision, semantic segmentation, and object detection are distinct yet crucial tasks. Object detection is an object-level task, while segmentation is a pixel-level task. Combining detection and segmentation tasks in a single network is challenging in feature sharing and task conflict handling. Moreover, shared features from the encoder might not optimally serve both tasks due to their different needs. The detection task focuses more on global information, while the segmentation task pays more attention to detailed information. Directly using the feature map from the detection head for segmentation prediction may lead to information confusion, increasing training difficulty.

We adopt the decoupled feature map as the detection and segmentation head input to address this challenge. In the segmentation branch, depicted in Fig. 2, we integrate C3

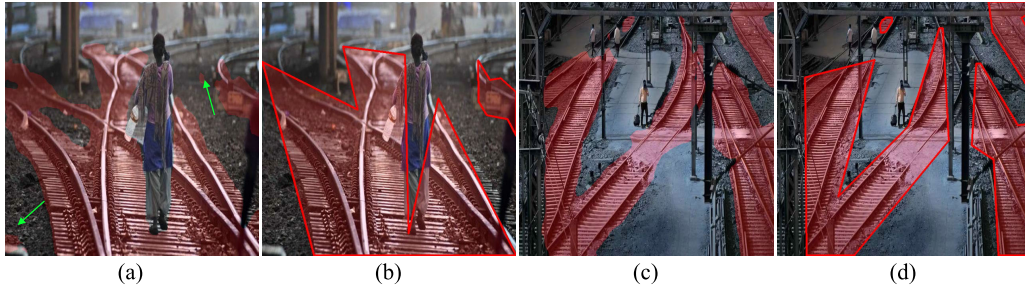


Fig. 3. Visualization of track masks before post-processing ((a) and (c)) and after post-processing ((b) and (d)) (green arrows point to noise).

and CSPP modules to enhance the feature and then recover the output feature map to the original size through bilinear interpolation. The output channels correspond to the number of segmentation classes, where each channel stores the probability that each pixel belongs to its corresponding class. Meanwhile, to meet various application scenarios and performance demands, we designed three simple segmentation decoder structures: S, M, and L. Let H be the height of the input image and W be the width of the input image. Specifically, S and M decode the predictions from the feature map with sizes of $\{H/16, W/16\}$ and $\{H/8, W/8\}$, respectively, containing both low-level and high-level features. L fuses feature maps of size $\{H/4, W/4\}$ and $\{H/8, W/8\}$ before prediction, containing richer details and global context information. We define the proposed railway multitask perception model with S, M, and L segmentation heads as MTP-Rail-S, MTP-Rail-M, and MTP-Rail-L, respectively.

C. Post-Processing

After identifying the track area and objects of interest using the multitask model, further post-processing is required to determine the presence of obstacles in the track area and classify the risk level. First, morphological preprocessing is applied to the segmented masks. Opening operations are used to remove noise, while closing operations connect gaps and fill holes, refining the mask information. Next, contours are extracted from the masks after morphological processing. To eliminate false detections, contours with areas smaller than a specified threshold are removed. These discarded contours typically represent noise or other insignificant details. For the remaining contours, their minimum external polygons are calculated and defined as track intrusion boundaries, which solves the problem of the mask edges obtained by model segmentation not being smooth.

Fig. 3 illustrates the before-and-after comparison of mask post-processing. We can see that post-processing effectively removes some of the noise (green arrows). It is worth noting that part of the grade crossing was not well segmented with the tracks in Fig. 3(a). This can occur due to the limitations of pixel-based segmentation algorithms, as well as the influence of complex backgrounds, varying lighting conditions, or image blur. However, the primary goal of track segmentation is to identify the intrusion boundary, rather than achieving perfect pixel-level accuracy. Post-processing can largely ignore these poorly segmented areas (as shown in Fig. 3(b)), allowing

the model to focus on the intrusion boundary. This approach enhances the model's robustness, ensuring reliable detection even in challenging conditions.

Finally, the distance and overlap between these railway intrusion boundaries and the detected bounding boxes of objects are checked to assess the intrusion risk. Three risk levels are defined: high risk (2) - inside the rails; low risk (1) - within the danger distance from the rails; safe (0) - outside the danger distance from the rails. Detailed pseudo code is provided in Algorithm 1. Here, d_{thresh} is a hyper-parameter used to measure the distance between the intrusion boundary and the detected object. According to the perspective principle, the farther the object is, the harder it is to define d_{thresh} . We set the default value of d_{thresh} to 50.

Algorithm 1 Obstacle Risk Level Classification

Input: image
Output: RiskLevel $\in \{0, 1, 2\}$.

- 1: Initialization: RiskLevel = 0, $d_{thresh} = 50$
- 2: bboxes, masks $\leftarrow$ Inference(image)
- 3: masks $\leftarrow$ Morphology_Preprocessing(masks)
- 4: contours = Find_Filter_Contours(masks)
- 5: **if not** contours.isEmpty() **and not** bboxes.isEmpty() **then**
- 6: **for** C in contours **do**
- 7: **if** Contour_Overlap_Boxes(C, bboxes) **then**
- 8: RiskLevel = 2
- 9: **break**
- 10: **else**
- 11: $d = \text{MinDistance_Contour_Boxes}(C, \text{bboxes})$
- 12: **if** $d \leq d_{thresh}$ **then**
- 13: RiskLevel = 2
- 14: **break**
- 15: **else**
- 16: tmpLevel = 1
- 17: RiskLevel = max(RiskLevel, tmpLevel)
- 18: **return** RiskLevel

D. Loss Function

Our loss consists of detection loss and segmentation loss. For detection, the loss function follows the YOLOv5, including classification loss L_{cls} , localization loss L_{CIoU} and confidence loss L_{conf} . Classification loss is used to measure the accuracy of object category classification by Binary Cross-Entropy (BCE). Localization loss employs Complete Intersection over Union (CIoU) to evaluate the alignment between the predicted bounding boxes and the ground truth. The confidence loss uses BCE to assess the confidence of the

object's presence within the box.

$$L_{cls}(y, \hat{y}) = - \sum_{i=1}^N \sum_{c=1}^C y_{ic} \log \hat{y}_{ic} \quad (1)$$

where N is the number of predicted bounding boxes, C is the number of classes, y_{ic} and $\hat{y}_{ic}$ are the ground truth and predicted probability of the c -th class of the i -th bounding box, respectively.

$$L_{conf} = - \frac{1}{N} \sum_i^N (y_i \log \hat{y}_i + (1 - y_i) \log (1 - \hat{y}_i)) \quad (2)$$

where y_i and $\hat{y}_i$ represent the actual label and predicted likelihood of the object in the i -th bounding box.

$$L_{CIoU} = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha v \quad (3a)$$

$$v = \frac{4}{\pi^2} (\arctan \frac{w^{gt}}{h^{gt}} - \arctan \frac{w}{h}) \quad (3b)$$

$$\alpha = \frac{v}{(1 - IoU) + v} \quad (3c)$$

where $\rho(b, b^{gt})$ is the Euclidean distance between the predicted box and ground truth. c is the diagonal length of the minimal outer box of the predicted box and the ground truth. w , h and w^{gt} , h^{gt} are the width and height of the predicted box and ground truth, respectively. v is a parameter used to measure the consistency of the aspect ratio of the predicted box and the ground truth. α is used to balance the impact of the aspect ratio consistency term on the total loss.

In segmentation, pixel-wise cross-entropy loss (CE) and Dice loss [48] are employed. CE measures the classification accuracy of each pixel, while Dice evaluates the overlap between the predicted mask and ground truth. The loss functions for semantic segmentation are defined as follows:

$$L_{CE} = - \frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{ic} \log \hat{y}_{ic} \quad (4)$$

where N is the total number of pixels, C denotes the number of segmentation classes, y_{ic} and $\hat{y}_{ic}$ represent the ground truth and predicted probability of the c -th class of the i -th pixel, respectively.

$$L_{Dice}(y, \hat{y}) = 1 - \frac{2 \sum_{i=1}^N y_i \hat{y}_i}{\sum_{i=1}^N y_i + \sum_{i=1}^N \hat{y}_i} \quad (5)$$

where N is the total number of pixels, y_i and $\hat{y}_i$ represent the ground truth and predictions at i -th pixel, respectively. In summary, the total loss function of the proposed multitask model is defined as follows:

$$L_{total} = \lambda_{cls} L_{cls} + \lambda_{loc} L_{CIoU} + \lambda_{conf} L_{conf} + \lambda_{CE} L_{CE} + \lambda_{Dice} L_{Dice} \quad (6)$$

where λ_{cls} , λ_{loc} , λ_{conf} , λ_{CE} , λ_{Dice} are weight parameters used to balance the loss.

E. Training Strategies

Deep learning is a data-driven approach, and the scale of the dataset directly impacts the model's ability to learn data distribution. Nevertheless, in railway intrusion detection, related data tend to be scarce, which may not meet the training and testing requirements. To address this challenge, transfer learning is employed to enhance the model performance. Transfer learning has been proven effective across multiple tasks, significantly improving model performance on the target domain. Specifically, we defined two auxiliary datasets as source domains for detection and segmentation tasks. Initially, the model is pre-trained on these auxiliary datasets to obtain initial training weights. Subsequently, during the training of the target model, these pre-trained weights are loaded for initialization, accelerating the convergence and enhancing its generalization capability.

We use End-to-End training strategy, multitask are trained jointly. However, influenced by YOLOP, we also attempted different training paradigms to optimize model performance, though experiments in Section IV-E1 show there is no significant improvement on our model. In most cases, training multiple tasks jointly may lead to instability, so the Step-by-Step training strategy was proposed. In other words, one task is trained first and then guide other tasks.

IV. EXPERIMENTS

In this section, we introduce the datasets and metrics we used and report the experimental results of the proposed multitask approach to validate its effectiveness and efficiency.

A. Datasets

1) *Railway Obstacle Detection (ROD) & Object-13*: ROD is an open-source railway obstacle detection dataset published on Roboflow [49]. To adapt this dataset for our multitask algorithm, we have augmented it by adding risk level annotations, segmentation annotations for the track area and adjusting the detection categories. Some of the type of railway obstacles we selected are inspired by [50]. The ROD only contains 300 images in total. Given that datasets containing potential obstacles in railway scenarios are scarce, they may be inadequate when directly employed for training and validation purposes. As a result, we established an extra object detection dataset, called Object-13, as the source domain for transfer learning to improve the accuracy of railway obstacle detection. Object-13 has a total of 800 images, mainly containing the categories of objects with insufficient samples in ROD. The object distributions in both the modified ROD dataset and the Object-13 are illustrated in Fig. 4.

2) *BDD100K*: BDD100K is a large-scale public driving dataset published by UC Berkeley that can be used for tasks such as object detection, and drivable area and lane segmentation. The dataset is diverse, covering different weather conditions, including sunny, cloudy, and rainy days, as well as day and night. However, the dataset has a high proportion of small objects, which makes detection and segmentation challenging. In this study, we reorganized the BDD100K to fit our multitask algorithm. Note that we have only kept five

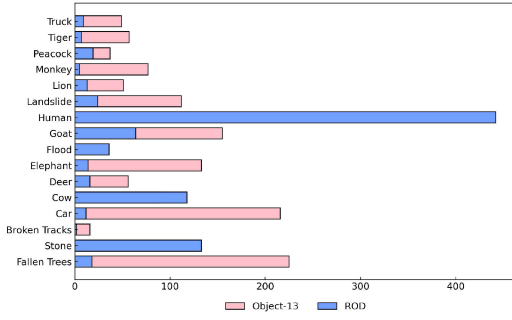


Fig. 4. Number of instances per class for ROD and Object-13.

classes: Car, Person, Rider, Traffic light, and Traffic sign out of the original 10 classes. Among them, Bus, Truck, and Train are merged into Cars, and Bike and Motor are merged into Rider. A total of 2976 customized BDD100k images were split into training and validation datasets according to a ratio of 9:1.

3) *Railsem19*: RailSem19 is a public dataset for railway semantic scene understanding, consisting of 8,500 images taken from the ego perspective of trains. It offers a wide range of semantic annotations, including geometric annotations (e.g., bounding box, polyline) and dense pixel-wise semantic labeling, which is compatible with Cityscapes. This study mainly uses this dataset to train and evaluate the segmentation performance of the multitask model. Only the track area is retained as the segmented foreground class.

B. Metrics

To assess the quality of the classification, we use the Precision (P), Recall (R), F1-score, and Overall Accuracy (OA). P measures the proportion of the correct positive predictions to the total number of positive predictions, and R is calculated as the number of correct positive predictions to the total number of positives. The F1 Score is useful to find an optimal balance between Precision and Recall. The proportion of correctly classified samples in total samples is OA, reflecting the overall performance of the classifier.

$$P = \frac{TP}{TP + FP} \quad (7)$$

$$R = \frac{TP}{TP + FN} \quad (8)$$

$$F1 = \frac{2PR}{P + R} \quad (9)$$

$$OA = \frac{TP + TN}{TP + FP + FN + TN} \quad (10)$$

where TP is True Positive, FP is False Positive, TN is True Negative, and FN is False Negative, respectively.

To comprehensively evaluate the performance of the multitask model, we assessed it from three aspects: detection accuracy, segmentation accuracy, and model complexity. For detection, the evaluation metrics usually include P, R, Average Precision (AP), and Mean Average Precision (mAP). A P-R curve can be plotted at different IoU confidence thresholds. The AP is obtained by calculating the area under the P-R curve (AUC) and measuring the detection performance for each class. mAP is the average of AP values across all classes

in a multi-class object detection task. The standard metric mAP@50 represents the performance at an IoU threshold of 0.5, calculated as follows.

$$AP = \int P(R) dR \quad (11)$$

$$mAP@50 = \frac{1}{C} \sum_{i=1}^C AP_i@50 \quad (12)$$

where C is the number of classes.

We adopt Mean Pixel Accuracy (mPA) and mean Intersection over Union (mIoU) as metrics for semantic segmentation evaluation. mPA is used to evaluate the proportion of correctly classified pixels. The higher the mPA, the more accurate the segmentation. mIoU measures the overlap between predicted regions and ground truth, ranging from 0 to 1.

$$mPA = \frac{1}{C+1} \sum_{i=0}^C \frac{TP + FN}{TP + FP + TN + FN} \quad (13)$$

$$mIoU = \frac{1}{C+1} \sum_{i=0}^C \frac{TP}{TP + FP + FN} \quad (14)$$

For model complexity, we primarily consider three evaluation metrics: parameters (#Param.), floating-point operations (FLOPs), and frames per second (FPS). FLOPs estimate the computing resources needed to execute the model. Models with higher FLOPs typically require more processing power and execution time. FPS indicates the number of input images processed per second and is commonly used to measure inference speed.

C. Implementation Details

All experiments were conducted on a workstation equipped with a 12th Gen Intel® Core™ i7-12700F@2.1GHz CPU and GeForce RTX 3060Ti with 8 GB of GDDR6 memory. The code is implemented by PyThon 3.9, PyTorch 1.13.1, and CUDA 11.7. During training, we utilized the stochastic gradient descent (SGD) algorithm with 0.95 momentum to train our model. We employed a 0.01 initial learning rate, 100 epochs, a batch size of 16, and an input resolution of 640×640 for all datasets. For data augmentation, we adopted several standard techniques, including image normalization, random horizontal flipping, color jittering, affine transformation, and disabled mosaic during the validation phase. We inherited the other hyperparameters from YOLOv5 to ensure consistency with its default settings. Given that we aimed to ensure the model's generalization and robustness without overly relying on fine-tuned parameters. The additional hyperparameters introduced in our model were empirically selected through handcrafting without extensive optimization methods such as grid search or Bayesian optimization. Specifically, the default values for the added hyperparameters were set as $d_{thresh} = 50$, $\lambda_{CE} = 0.05$, $\lambda_{Dice} = 0.5$.

D. Comparison Experiment

This section validates the effectiveness of the proposed multitask model through comparative experiments, which

TABLE II
RESULTS OF RAILWAY OBJECT DETECTION ON ROD: COMPARING THE PROPOSED MODEL WITH STATE-OF-THE-ART DETECTORS

Model	#Param. (M)	FLOPs (G)	P	R	mAP	FPS
SSD	25.75	62.35	34.54%	54.35%	52.9%	107
YOLOv5-S	7.05	15.9	57.6%	43.4%	47.9%	227
YOLOv9-T	2.62	10.7	51.6%	44.4%	43.7%	200
YOLOv9-S	9.61	38.8	49.7%	55%	57.8%	110
YOLOv9-M	32.57	130.8	77.1%	45.8%	56%	50
YOLOv10-N	2.70	8.3	69.5%	40.4%	48.3%	256
YOLOv10-S	8.04	24.5	69%	50.4%	55.9%	156
YOLOv10-M	16.5	64.1	52%	43.3%	46.1%	83
MTP-Rail-S (Ours)	7.67	17.9	64.3%	49.2%	51.3%	227
MTP-Rail-M (Ours)	7.67	24.1	64.2%	43.5%	49.5%	181
MTP-Rail-L (Ours)	7.75	53.1	65.9%	47.2%	50.5%	87

Note: S, M, T and N indicate model scales: S (Small), M (Medium), T (Tiny), N (Nano).

TABLE III
RESULTS OF RAILWAY SEMANTIC SEGMENTATION ON ROD: COMPARING THE PROPOSED MODEL WITH A STATE-OF-THE-ART METHOD

Model	FLOPs (G)	mPA	mIoU	FPS
BiSeNetv2	19.3	82.3%	69.4%	138
MTP-Rail-S	17.9	80.7%	69.5%	227
MTP-Rail-M	24.1	80.5%	69.3%	181
MTP-Rail-L	53.1	79.9%	69.7%	87

TABLE IV
SINGLE CLASS OBJECT DETECTION RESULTS ON BDD100k: COMPARING THE PROPOSED MODEL WITH YOLOP

Model	mAP	mPA	mIoU	FPS
YOLOP	76.5%	73.6%	57.3%	152
MTP-Rail-S	73.7%	68.3%	49.6%	227
MTP-Rail-M	73.8%	74.4%	56.8%	181
MTP-Rail-L	73.7%	77.2%	59.9%	87

mainly include object detection and segmentation on ROD and BDD100k, as well as railway obstacle classification on ROD. In addition to comparing the proposed model with the state-of-the-art multi-task perception model YOLOP, it is also compared to advanced real-time single-task detection models (e.g., SSD, YOLOv5, YOLOv9, YOLOv10) and segmentation models (e.g., BiSeNetv2) separately, although this is not a fair comparison for our algorithm. By the way, all models in this section are trained end-to-end.

1) *Object Detection Results*: Table II compares our proposed MTP-Rail with other state-of-the-art real-time object detectors on ROD, although it would be unfair to compare them with dedicated detectors. Except for SSD, the #Param. and FLOPs of models are calculated when the input size is 640×640 . The input size for SSD is 300×300 . YOLOv9-S achieves the best mAP of 57.8% with 110 FPS. Compared with similarly lightweight models such as YOLOv5-S, YOLOv9-T [51], and YOLOv10-N [52], the proposed MTP-Rail-S has a comparable inference speed but has a 2.4% ~ 7.6% improvement in mAP. The real-time and high-accuracy characteristics of MTP-Rail-S fully demonstrate its practical application potential in real-world railway monitoring systems. Note that the proposed MTP-Rail-{S, M, L} mainly differs in the segmentation head, and thus the detection results on small datasets are not very different. To evaluate the detection performance of MTP-Rail more reliably, we will test it on BDD100k later due to the limited sample size of ROD.

2) *Railway Track Segmentation Results*: Similarly, we compare the proposed MTP-Rail with another state-of-the-art real-time semantic segmentation method BiSeNetv2 on ROD, see Table III. Compared with the specialized segmentation approach BiSeNetv2, MTP-Rail-{S, M, L} shows slightly lower performance in terms of mPA, and mIoU. The most likely reason for this result is that on a small dataset, the performance difference of MTP-Rail-{S, M, L} is not obvious.

3) *Multitask Results*: Given the limited number of samples and the absence of a public baseline for multitask for railway scenes, we opted to utilize the well-known BDD100K to assess the performance of our multitask algorithm further. YOLOP, a state-of-the-art multitask perception model, is capable of concurrently handling three tasks: object detection, drivable area segmentation, and lane segmentation, demonstrating excellent performance in terms of speed and accuracy. Therefore, in this section, we aim to compare YOLOP with our proposed model under single-class and multi-class object detection scenarios. The difference between these scenarios lies in the classes to be detected while the segmentation classes are the same. Details are below.

- **Single-class**: Car, Bus, Truck and Train are combined into one class and are denoted as Car to compare the effect of single-class detection. The experimental results are shown in Table IV. We can see that YOLOP performs well in detection and demonstrates comparable performance to our approach in segmentation, but our algorithm significantly outperforms YOLOP in terms of speed.
- **Multi-class**: we perform multi-class object detection, including Car, Person, Rider, Traffic light, and Traffic sign classes. The results, as presented in Table V, indicate that our algorithm leads YOLOP in detection accuracy across all classes in the multi-class object detection task. It is worth noting that our average accuracy is almost twice that of YOLOP, which further demonstrates the superiority of our multitask model in multi-class object detection.

Table VI further demonstrates the semantic segmentation result of YOLOP and the proposed model on BDD100K. Our model surpasses YOLOP in all metrics in drivable area segmentation, especially in terms of IoU. In lane segmentation, the segmentation accuracy of YOLOP is higher than MTP-Rail-S and MTP-Rail-M, but almost indifferent to MTP-Rail-L. YOLOP introduces two separate segmentation heads

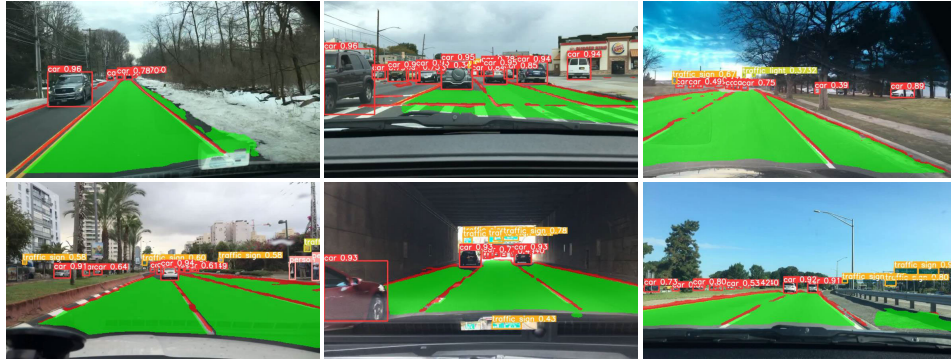


Fig. 5. Visualization of prediction results on BDD100K.

TABLE V

MULTI-CLASS OBJECT DETECTION RESULTS ON BDD100k: COMPARING THE PROPOSED MODEL WITH YOLOP

	Car	Person	Rider	Traffic sign	Traffic light	mAP
YOLOP	45.1%	18.3%	7.5%	19.7%	30.5%	24.2%
MTP-Rail-S	73.7%	52.2%	34.0%	47.4%	53.3%	52.1%
MTP-Rail-M	73.8%	50.4%	37.9%	48.2%	51.8%	52.4%
MTP-Rail-L	73.7%	50.6%	33.8%	47.8%	52.9%	51.8%

TABLE VI

SEMANTIC SEGMENTATION RESULTS ON BDD100k: COMPARING THE PROPOSED MODEL WITH YOLOP

Model	CPA		mPA	IoU		mIoU
	Drivable Area	Lane		Drivable Area	Lane	
YOLOP	84.2%	63.0%	73.6%	76.3%	38.3%	57.3%
MTP-Rail-S	86.4%	50.2%	68.3%	80.4%	18.7%	49.6%
MTP-Rail-M	87.6%	59.1%	73.4%	81.8%	31.7%	56.8%
MTP-Rail-L	87.9%	66.4%	77.2%	82.2%	37.6%	59.9%

to predict the drivable area and lane, achieving enhanced segmentation accuracy with fewer lane pixels but increasing the model complexity. Notably, the proposed MTP-Rail-L with the highest model complexity reaches an impressive inference speed of 87 FPS and still meets the real-time requirement. As expected, there are obvious performance gaps among MTP-Rail-{S, M, L} with different complexity levels on BDD100K. Overall, the proposed algorithm demonstrates superior overall performance, striking an optimal balance between accuracy and speed and demonstrating excellent practical value. For instance, in railway monitoring, its ability to detect objects and drivable areas simultaneously makes it ideal for level crossing management, enabling early hazard detection. Moreover, the end-to-end training approach simplifies both training and inference processes, while the lightweight design supports efficient deployment on edge devices, making it well-suited for real-time safety monitoring and automated maintenance.

Fig. 5 and Fig. 6 give the prediction results of the proposed method on BDD100K and ROD, respectively. It is clear that the proposed method achieves satisfactory performance in both object detection and semantic segmentation. In addition, it has strong robustness and generalization ability, performing well on different datasets.

4) *Obstacle Classification Results:* It is essential to distinguish the actual obstacles of the detection and segmentation outputs from the multitask model, which alleviates the false positives and enhances the obstacle recognition. Refined post-processing removes voids and reconnects fragmented sections within the track area. The boundary of the track area is

TABLE VII

RESULTS OF THE PROPOSED POST-PROCESSING METHOD FOR OBSTACLE RISK LEVEL CLASSIFICATION

	P	R	F1-Score	OA
Safe	69.2%	83.3%	75.6%	89.6%
Low	66.7%	37.5%	48.0%	
High	94.7%	93.4%	94.0%	
Macro-avg	76.9%	71.4%	72.6%	

defined by extracting the minimum track external polygon of the connected region. Fig. 8 visualizes the obstacle risk level classification results. Table VII quantifies the results of the proposed post-processing method for obstacle risk level classification on ROD. The results indicate that the post-processing algorithm performs excellently at high-risk levels scene, with 94.7% precision. However, regarding low-risk level scenes, the model performs poorly with only 48.0% F1 score, resulting in a high missed detection and false alarm. We introduced the relative distance threshold d_{thresh} between the track area and objects to better classify the risk level. According to the perspective projection, a set of parallel railway tracks converges to a vanishing point in the distance. Thus, d_{thresh} is considered a distance-sensitive hyperparameter, and its values significantly impact the results. In other words, this non-automated hyperparameter may be the main bottleneck limiting the accuracy of safe and low-risk area determination. To observe the classification performance across different risk levels straightforwardly, we plotted the confusion matrix, as shown in Fig. 7.



Fig. 6. Visualization of prediction results on ROD.

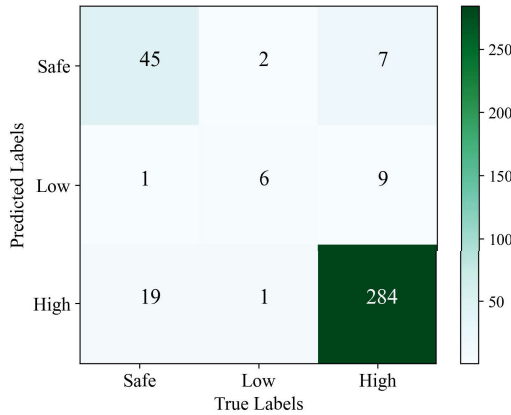


Fig. 7. Confusion matrix of risk level classification.

E. Ablation Experiment

In this section, we set up several ablation experiments for the proposed method to further analyze which training strategies or model components play a key role in improving the model performance.

1) *Training Schemes*: Due to the limited sample size of the rail transit dataset, the data available for each task is significantly less than that for general scenarios, making it challenging to fully validate the impact of different training strategies on the model. Therefore, in this section, we adopted MTP-Rail-M on BDD100K to explore the effects of different training strategies on model performance. Specifically, we analyze the following two cases: 1) single-task learning vs. multitask learning, and 2) End-to-End training vs. Step-by-Step training. The training modes for mere detection and mere segmentation tasks are called Det-only and Seg-only, respectively. Training the model first in Det-only mode and then loading the trained weights for end-to-end training is termed Det-first, while the opposite order is Seg-first. End-to-End represents the training of detection and segmentation tasks simultaneously. The experimental results are presented in Table VIII.

It is apparent that the single-task training performs the best on individual tasks, achieving 53.1% mAP for the detection task and 76.9% mPA for the semantic segmentation task.

TABLE VIII

PERFORMANCE COMPARISON OF DIFFERENT TRAINING SCHEME

Training method	Detection	Semantic Segmentation	
	mAP	mPA	mIoU
Det-only	53.1%	-	-
Seg-only	-	76.9%	61.4%
Det-first	51.3%	73.6%	55%
Seg-first	52%	74.8%	58.1%
End-to-End	52.4%	73.4%	56.8%

Compared to End-to-End, the Seg-first shows a slight reduction in detection accuracy but a boost in semantic segmentation accuracy. Det-first not only suffers from a drop in detection accuracy but also has the lowest segmentation accuracy. The possible reason is that using Dep-first the model needs to adjust the already learned weights to adapt to the new task during the second training phase, which may be less generalizable than End-to-End training. Although the End-to-End mode does not achieve the best performance in each task, it strikes a better balance between detection and segmentation tasks. Overall, the proposed model demonstrates robust learning abilities in multitasking, and the training order of different tasks has little impact on performance. Considering the simplicity of training and model performance, the End-to-End mode is undoubtedly a more competitive choice.

2) *Transfer Learning*: A small number of ROD training samples will degrade the accuracy of detection and segmentation, so we use transfer learning to improve the model performance. Object13 and Railsem19 are used as the source domain for detection and segmentation, respectively, and ROD is used as the target domain. The first step of this experiment is to train the source data in single-task mode to get the initial weight parameters. Then, end-to-end training with pre-trained weights will be performed separately on ROD. We also train and test the model on ROD while loading pre-trained detection and segmentation weights from the source domain at the same time. We set the baseline by doing end-to-end training without transfer learning. All experiments in this section were conducted using MTP-Rail-M; see results in Table IX.

The results indicate that initializing the model with Object13 training weights can improve the detection mAP by 4.8% compared with the baseline, while the segmentation accuracy

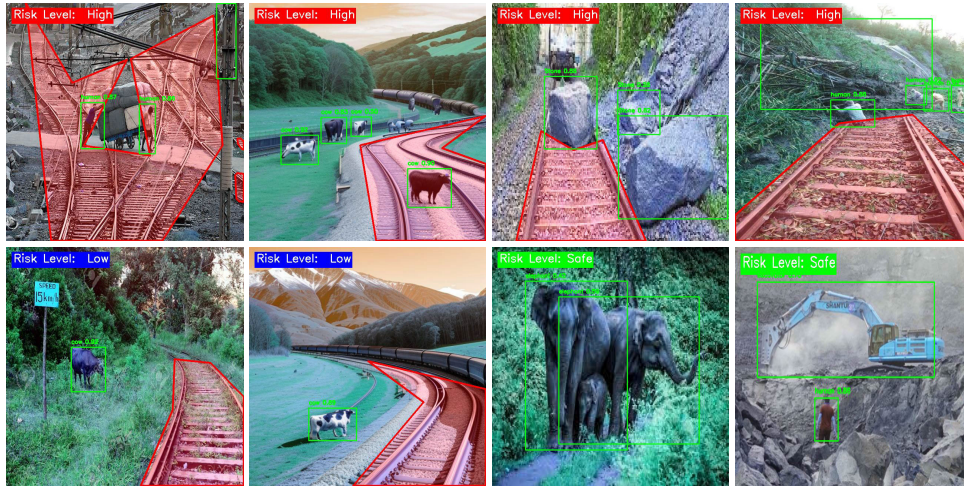


Fig. 8. Visualization of obstacle risk level classification results.

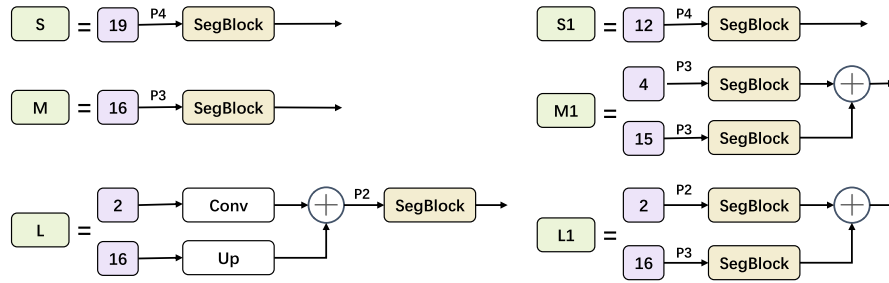


Fig. 9. Illustration of different segmentation decoder structures.

TABLE IX

RESULTS OF SEMANTIC SEGMENTATION EXPERIMENTS. S* MEANS ONLY LOAD THE WEIGHTS FROM SEGMENTATION HEAD

Object-13	Railsem19	ROD	mAP	mPA	mIoU
		✓	49.5%	80.5%	69.3%
	✓		-	93.0%	85.8%
	✓	✓	38.9%	88.5%	80.2%
✓			62.9%	-	-
✓		✓	54.3%	78.6%	64.9%
✓	✓ S*	✓	53.3%	79.3%	66.9%

decreases to varying degrees. Initializing the model with Railsem19 training weights improves segmentation mPA and mIoU by 8% and 10.9%, respectively, but detection accuracy drops. Detection and segmentation share the same encoder in our model. When loading pre-trained weights from object-13 and railsem19, we needed to select the appropriate encoder weights. The shape of the detection head from object-13 did not match our model due to the different number of classes, making it incompatible for direct loading. As a result, we decided to load the encoder and detection head weights from object-13 and the segmentation head weights from railsem19 into our model. This combination offers a balanced approach, making it suitable for applications requiring both detection and segmentation capabilities. In summary, transfer learning can effectively alleviate the accuracy decline caused by the limited scale of the ROD dataset.

3) *Segmentation Head*: To investigate the impact of multi-scale features and feature fusion methods on segmentation tasks, we conduct ablation studies on the segmentation head.

We have extended the proposed semantic segmentation head with S, M, and L structures by introducing three new structures, namely S1, M1, and L1. The specific configurations are illustrated in Fig. 9. Specifically, the S and S1 heads employ single-scale features $\{H/16, W/16\}$ for semantic segmentation decoding. Both M and M1 heads predict the features with input size $\{H/8, W/8\}$, but the former fuses different features and then predicts mask, while the latter predicts masks first and then performs feature fusion. The L and L1 heads predict features with input sizes $\{H/4, W/4\}$ and $\{H/8, W/8\}$, respectively, but the former predicts masks after feature fusion, and the latter predicts masks before feature fusion.

We carried out experiments on ROD and BDD100k, respectively, and the experimental results are listed in Table X. It shows that the S and S1 heads have the lowest computational complexity and the fastest inference speed, up to 230 FPS, when the unified input size is 640×640 . However, they lag slightly behind other schemes in terms of segmentation accuracy. The segmentation performance of M and M1 head is basically the same, but M performs better in detection, and inference speed is faster, reaching 181 FPS. The L and L1 heads have the highest model complexity and the slowest inference speed, about 80 FPS, but still meet the real-time requirements. The segmentation performance of prediction after fusion scheme L is obviously better than that of the prediction before fusion scheme L1 on BDD100k, while the two are similar on the ROD dataset. This could be attributed to the small size of the ROD dataset, which has a relatively more uniform object scale, making the data reasonably easy to fit.

TABLE X
PERFORMANCE OF DIFFERENT SEMANTIC SEGMENTATION DECODERS ON BDD100K AND ROD

Seg-Head	#Param. (M)	FLOPs@640 (G)	FPS	BDD100K			ROD		
				mAP	mPA	mIoU	mAP	mPA	mIoU
L1	8.26	54.5	81	52.1%	71.6%	57.9%	53.6%	80%	66.5%
L	7.75	53.1	87	51.8%	77.2%	59.9%	51.1%	80.7%	66.1%
M1	8.25	31.5	131	52.1%	73.7%	55.6%	51.7%	78.8%	64.3%
M	7.67	24.1	181	52.4%	73.4%	56.8%	54.3%	78.6%	64.9%
S1	7.73	18.1	231	52.8%	67.7%	48.5%	51.7%	78.9%	66.4%
S	7.67	17.9	227	52.1%	68.3%	49.6%	51.9%	79.7%	66.4%

Consequently, employing a complex model may inadvertently increase the risk of overfitting.

In summary, different segmentation heads exhibit a marginal impact on detection. Nonetheless, it remains evident that complex segmentation heads can influence model fitting, posing challenges to balance detection and segmentation tasks. According to the specific application demands and resource constraints, choosing the appropriate Seg-Head scheme will help to achieve the best semantic segmentation performance. Overall, the M-head achieves a best balance between accuracy and speed.

V. CONCLUSION

In this paper, we applied emerging technologies to develop a real-time end-to-end railway obstacle detection model that can simultaneously detect objects, extract track areas, and assess obstacle risk levels. To the best of our knowledge, the proposed approach is the first open-source end-to-end railway obstacle intrusion detection model based on multitask perception, which achieves excellent performance in terms of both speed and accuracy and has the potential for real-world applications. Our model exhibits good generalization ability and achieves advanced performance on the comprehensive BDD100k dataset. In addition, we conduct extensive experiments to verify the algorithm's effectiveness.

Nevertheless, our method still has several limitations. It does not consider the need for long-range object detection, which is crucial for enhancing system safety. Additionally, as our algorithm relies on supervised learning, it struggles to identify obstacles outside the training data. Moreover, our vision-based model may not perform well in harsh environments such as nighttime, rain, snow, and high-speed conditions. Future research will focus on addressing these limitations to enhance the model's overall performance and application scope. Specifically, we aim to optimize long-range detection, explore open-world detection methods, and improve robustness in challenging environments.

REFERENCES

- [1] D. Ristić-Durrant, M. Franke, and K. Michels, "A review of vision-based on-board obstacle detection and distance estimation in railways," *Sensors*, vol. 21, no. 10, p. 3452, May 2021.
- [2] Z. Xie and Y. Qin, "High-speed railway perimeter intrusion detection approach based on Internet of Things," *Adv. Mech. Eng.*, vol. 11, no. 2, Feb. 2019, Art. no. 1687814018821511.
- [3] P. Singh, M. A. Dulebenets, J. Pasha, E. D. R. S. Gonzalez, Y.-Y. Lau, and R. Kampmann, "Deployment of autonomous trains in rail transportation: Current trends and existing challenges," *IEEE Access*, vol. 9, pp. 91427–91461, 2021.
- [4] J. Canny, "A computational approach to edge detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. PAMI-8, no. 6, pp. 679–698, Nov. 1986.
- [5] R. O. Duda and P. E. Hart, "Use of the Hough transformation to detect lines and curves in pictures," *Commun. ACM*, vol. 15, no. 1, pp. 11–15, 1972.
- [6] W. Liu et al., "SSD: Single shot multibox detector," in *Proc. 14th Eur. Conf. Comput. Vis.*, Amsterdam, The Netherlands: Cham, Switzerland: Springer, Oct. 2016, pp. 21–37.
- [7] P. Jiang, D. Ergu, F. Liu, Y. Cai, and B. Ma, "A review of YOLO algorithm developments," *Proc. Comput. Sci.*, vol. 199, pp. 1066–1073, Jan. 2022.
- [8] R. Girshick, "Fast R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Dec. 2015, pp. 1440–1448.
- [9] T. Lin et al., "Microsoft COCO: Common objects in context," in *Proc. Eur. Conf. Comput. Vis.*, Zurich, Switzerland: Cham, Switzerland: Springer, Sep. 2014, pp. 740–755.
- [10] H. Caesar et al., "NuScenes: A multimodal dataset for autonomous driving," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 11621–11631.
- [11] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2012, pp. 3354–3361.
- [12] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2980–2988.
- [13] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2961–2969.
- [14] C. Yu, J. Wang, C. Peng, C. Gao, G. Yu, and N. Sang, "BiseNet: Bilateral segmentation network for real-time semantic segmentation," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Sep. 2018, pp. 325–341.
- [15] M. Franke, V. Gopinath, D. Ristić-Durrant, and K. Michels, "Object-level data augmentation for deep learning-based obstacle detection in railways," *Appl. Sci.*, vol. 12, no. 20, p. 10625, Oct. 2022.
- [16] S. Perić, M. Milojković, S.-D. Stan, M. Banić, and D. Antić, "Dealing with low quality images in railway obstacle detection system," *Appl. Sci.*, vol. 12, no. 6, p. 3041, Mar. 2022.
- [17] M. Banić et al., "Prediction of dynamic response of vibration isolated railway obstacle detection system," *Acta Polytechnica Hungarica*, vol. 19, no. 3, pp. 51–64, 2022.
- [18] R. Tian, H. Shi, B. Guo, and L. Zhu, "Multi-scale object detection for high-speed railway clearance intrusion," *Int. J. Speech Technol.*, vol. 52, no. 4, pp. 3511–3526, Mar. 2022.
- [19] M. Guo et al., "Attention mechanisms in computer vision: A survey," *Comput. Vis. Media*, vol. 8, no. 3, pp. 331–368, 2022.
- [20] Z. Zhang, P. Chen, Y. Huang, L. Dai, F. Xu, and H. Hu, "Railway obstacle intrusion warning mechanism integrating YOLO-based detection and risk assessment," *J. Ind. Inf. Integr.*, vol. 38, Mar. 2024, Art. no. 100571.
- [21] M. Klammsteiner, M. Döller, P. Golec, M. Kohlegger, S. Mayr, and E. Rashid, "Vision based stationary railway track monitoring system," in *Proc. 33rd Conf. Open Innov. Assoc. (FRUCT)*, May 2023, pp. 325–330.
- [22] W. Zhangyu, Y. Guizhen, W. Xinkai, L. Haoran, and L. Da, "A camera and LiDAR data fusion method for railway object detection," *IEEE Sensors J.*, vol. 21, no. 12, pp. 13442–13454, Jun. 2021.
- [23] Z. Tang and J. Yang, "Research on active obstacle detection algorithm of rail train based on multi-sensor," in *Proc. 8th Int. Conf. Cloud Comput. Big Data Analytics (ICCCBDA)*, Apr. 2023, pp. 556–559.
- [24] J. Qu, S. Li, Y. Li, and L. Liu, "Research on railway obstacle detection method based on developed Euclidean clustering," *Electronics*, vol. 12, no. 5, p. 1175, Feb. 2023.
- [25] X. Yu and M. Marinov, "A study on recent developments and issues with obstacle detection systems for automated vehicles," *Sustainability*, vol. 12, no. 8, p. 3281, Apr. 2020.

- [26] F. Maire and A. Bigdeli, "Obstacle-free range determination for rail track maintenance vehicles," in *Proc. 11th Int. Conf. Control Autom. Robot. Vis.*, Dec. 2010, pp. 2172–2178.
- [27] H. Wang, X. Zhang, L. Damiani, P. Giribone, R. Revetria, and G. Ronchetti, "Transportation safety improvements through video analysis: An application of obstacles and collision detection applied to railways and roads," in *Proc. Trans. Eng. Technol., Int. MultiConf. Eng. Comput. Scientists*. Cham, Switzerland: Springer, Jan. 2018, pp. 1–15.
- [28] R. Wang, Y. Chen, and H. Wang, "Research on environment perception and obstacle detection for unmanned vehicle based on machine vision," in *Proc. Int. Conf. Electr. Inf. Technol. Rail Transp.* Cham, Switzerland: Springer, Jan. 2020, pp. 565–575.
- [29] T. Lu, C. Geng, and H. Dai, "Subway obstacle detection system based on machine vision," in *Proc. Int. Conf. Intell. Transp. Eng.* Cham, Switzerland: Springer, Jan. 2022, pp. 1115–1122.
- [30] X. Gong, X. Chen, Z. Zhong, and W. Chen, "Enhanced few-shot learning for intrusion detection in railway video surveillance," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 8, pp. 11301–11313, Aug. 2021.
- [31] F. U. Rahman, M. T. Ahmed, M. M. Hasan, and N. Jahan, "Real-time obstacle detection over railway track using deep neural networks," *Proc. Comput. Sci.*, vol. 215, pp. 289–298, Jul. 2022.
- [32] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 4510–4520.
- [33] G. Jocher et al. *Ultralytics/YOLOv5*. (V6.2). Github. Accessed: Aug. 21, 2020. [Online]. Available: <https://github.com/ultralytics/yolov5>
- [34] C. Yu, C. Gao, J. Wang, G. Yu, C. Shen, and N. Sang, "BiSeNet v2: Bilateral network with guided aggregation for real-time semantic segmentation," *Int. J. Comput. Vis.*, vol. 129, no. 11, pp. 3051–3068, Nov. 2021.
- [35] T. Ye, C. Ren, X. Zhang, G. Zhai, and R. Wang, "Application of lightweight railway transit object detector," *IEEE Trans. Ind. Electron.*, vol. 68, no. 10, pp. 10269–10280, Oct. 2021.
- [36] Y.-M. Sun, Z. Xie, Y. Qin, L. Chuan, and Z. Wu, "Image detection of foreign body intrusion in railway perimeter based on dual recognition method," in *Proc. Eur. Workshop Struct. Health Monit., Special Collection Papers-Volume*. Cham, Switzerland: Springer, Jan. 2021, pp. 645–654.
- [37] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," 2018, *arXiv:1804.02767*.
- [38] H. Qin, A. Zaman, and X. Liu, "Artificial intelligence-aided intelligent obstacle and trespasser detection based on locomotive-mounted forward-facing camera data," *Proc. Inst. Mech. Eng., F. J. Rail Rapid Transit*, vol. 237, no. 9, pp. 1230–1241, Oct. 2023.
- [39] S. Li, H. Zhao, and J. Ma, "An edge computing-enabled train obstacle detection method based on YOLOv3," *Wireless Commun. Mobile Comput.*, vol. 2021, pp. 1–9, Oct. 2021.
- [40] L. Guan, L. Jia, Z. Xie, and C. Yin, "A lightweight framework for obstacle detection in the railway image based on fast region proposal and improved YOLO-tiny network," *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1–16, 2022.
- [41] Z. Cao et al., "An effective railway intrusion detection method using dynamic intrusion region and lightweight neural network," *Measurement*, vol. 191, Mar. 2022, Art. no. 110564.
- [42] A. Boussik, W. Ben-Messaoud, S. Niar, and A. Taleb-Ahmed, "Railway obstacle detection using unsupervised learning: An exploratory study," in *Proc. IEEE Intell. Vehicles Symp. (IV)*, Jul. 2021, pp. 660–667.
- [43] M. Brucker, A. Cramariuc, C. Von Einem, R. Siegwart, and C. Cadena, "Local and global information in obstacle detection on railway tracks," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2023, pp. 9049–9056.
- [44] H. Pan, Y. Li, H. Wang, and X. Tian, "Railway obstacle intrusion detection based on convolution neural network multitask learning," *Electronics*, vol. 11, no. 17, p. 2697, Aug. 2022.
- [45] Y. Tang, Y. Wang, and Y. Qian, "Railroad crossing surveillance and foreground extraction network: Weakly supervised artificial-intelligence approach," *Transp. Res. Rec., J. Transp. Res. Board*, vol. 2677, no. 9, pp. 525–538, Sep. 2023.
- [46] D. Wu et al., "YOLOP: You only look once for panoptic driving perception," *Mach. Intell. Res.*, vol. 19, no. 6, pp. 550–562, Dec. 2022.
- [47] D. Vu, B. Ngo, and H. Phan, "HybridNets: End-to-end perception network," 2022, *arXiv:2203.09035*.
- [48] F. Milletari, N. Navab, and S. Ahmadi, "V-Net: Fully convolutional neural networks for volumetric medical image segmentation," in *Proc. 4th Int. Conf. 3D Vis. (3DV)*, Oct. 2016, pp. 565–571.
- [49] T. Proj. (Nov. 2023). *Obstacle Detection Dataset*. Accessed: Jul. 7, 2024. [Online]. Available: <https://universe.roboflow.com/tarp-proj/obstacle-detection-t5lua>
- [50] Y. Huang, Z. Zhang, Y. Tao, and H. Hu, "Quantitative risk assessment of railway intrusions with text mining and fuzzy rule-based bow-tie model," *Adv. Eng. Informat.*, vol. 54, Oct. 2022, Art. no. 101726.
- [51] C.-Y. Wang, I.-H. Yeh, and H.-Y. Mark Liao, "YOLOv9: Learning what you want to learn using programmable gradient information," 2024, *arXiv:2402.13616*.
- [52] A. Wang et al., "YOLOv10: Real-time end-to-end object detection," 2024, *arXiv:2405.14458*.



Chenglin Chen received the B.E. degree from Shandong University of Technology, Shandong, in 2019, and the M.S. degree from Beijing Institute of Technology, Beijing, in 2022. She is currently pursuing the Ph.D. degree in intelligent transportation with The Hong Kong University of Science and Technology (Guangzhou). Her current research interests include computer vision, railway operation safety, computing methods, and 3D reconstruction.



Huixiong Qin received the B.E. degree from Sichuan University in 2019 and the M.S. degree from Rutgers University in 2020, where he is currently pursuing the Ph.D. degree. His research interests include developing machine learning infrastructure and applying computer vision techniques to improve transportation safety.



Yong Qin (Senior Member, IEEE) received the Ph.D. degree from China Academy of Railway Sciences, Beijing, China, in 1999. He is currently with the State Key Laboratory of Rail Traffic Control and Safety, Beijing Jiaotong University, Beijing. His current research interests include area of intelligent transportation systems, railway operation safety, and reliability.



Yun Bai received the Ph.D. degree from the University of Illinois at Urbana-Champaign, Champaign, IL, USA, in 2013. This work was partially done when she was an Assistant Professor with the Intelligent Transportation Thrust, The Hong Kong University of Science and Technology (Guangzhou). She is currently an Assistant Professor with New Jersey Institute of Technology. Her recent research interests include rail transit infrastructure management applying and customizing the state-of-the-art AI, sensing, and computing methods and technologies.